Package ‘replayr’

July 18, 2026

Title Read Emulator Movie Files into a Uniform Frame-by-Frame Array

Version 0.3.3

Description Decoders for twelve emulator movie (``input log'') file formats used by the speedrun and tool-assisted-speedrun communities: 'FCEUX' (.fm2), 'BizHawk' (.bk2), 'FCEU' (.fcm), 'lsnes' (.lsmv), 'Gens-rerecording' (.gmv), 'VBA-rerecording' (.vbm), 'Mupen64' (.m64), 'Snes9x-rerecording' (.smv), 'PCSX-rerecording' (.pxm), 'PSXjin' (.pjm), 'Dolphin' (.dtm), and 'Doom' (.lmp). Every
parser produces the same in-memory shape: a per-frame logical matrix indexed against a single 40-slot universal-button layout, plus the file's header metadata and a per-frame commands vector. Allows cross-platform analysis of input recordings without an emulator.

License GPL (>= 2)

Encoding UTF-8

Language en-US

Roxygen list(markdown = TRUE)

Depends R (>= 4.0)

Imports utils

Suggests knitr, rmarkdown, testthat (>= 3.0.0)

VignetteBuilder knitr

Config/testthat/edition 3

Config/roxygen2/version 8.0.0

NeedsCompilation no

Author Thorben Pelzer [aut, cre]

Maintainer Thorben Pelzer <thorben@pelzer.blog>

Contents

replayr-package 2
n_buttons 3
n_frames 3
n_players 4
player 4

press_counts . . . 5
read_bk2 . . . 5
read_dtm . . . 6
read_fcm . . . 6
read_fm2 . . . 7
read_gmv . . . 7
read_lmp . . . 8
read_lsmv . . . 9
read_m64 . . . 9
read_movie . . . 10
read_pjm . . . 10
read_pxm . . . 11
read_smv . . . 11
read_vbm . . . 12
replayr_movie . . . 12
supported_formats . . . 13
universal_buttons . . . 13
universal_button_names . . . 14

Index **15**

`replayr-package` *replayr: parsers for emulator movie ("input log") files*

Description

Decoders for 12 emulator movie file formats used by the speedrun and tool-assisted-speedrun communities. Every parser produces a uniform in-memory shape (a `replayr_movie` S3 object) with a per-frame logical input matrix indexed against a single 40-slot universal-button layout.

Details

Supported formats: .fm2 (FCEUX), .bk2 (BizHawk), .fcm (FCEU), .lsmv (lsnes), .gmv (Gens-rerecording), .vbm (VBA-rerecording), .m64 (Mupen64), .smv (Snes9x-rerecording), .pxm (PCSX-rerecording), .pjm (PSXjin text), .dtm (Dolphin GameCube), .lmp (Doom).

Quick start

```
library(replayr)
mv <- read_movie("data/raw_nes/super_mario_bros/1349.fm2")
n_frames(mv)                    # frame count
press_counts(mv)                # per-button press counts
p1 <- player(mv, 1L)            # logical matrix, frames x buttons
```

Universal button layout

Each `replayr_movie` indexes its third dimension against a prefix of the 40-slot universal layout. Slots 1..8 are NES-shaped (R, L, D, U, Start, Select, B, A); higher slots cover SNES, Genesis, GBA, N64, PSX, GameCube. See `universal_buttons()` and `universal_button_names()`.

Author(s)

Maintainer: Thorben Pelzer <thorben@pelzer.blog>

Authors:

- Thorben Pelzer <thorben@pelzer.blog>

`n_buttons` *Number of universal-button slots in a parsed movie*

Description

Varies per platform: 8 for NES / GB, 12 for SNES / GBA, 15 for Genesis, 28 for N64 / GameCube, 30 for PSX. Slots are always a prefix of the `universal_buttons()` alphabet.

Usage

```
n_buttons(movie)
```

Arguments

`movie`	A replayr_movie (returned by `read_movie()` or any of the format-specific readers).

Value

Integer scalar.

`n_frames` *Number of frames in a parsed movie*

Description

Number of frames in a parsed movie

Usage

```
n_frames(movie)
```

Arguments

`movie`	A replayr_movie (returned by `read_movie()` or any of the format-specific readers).

Value

Integer scalar: the first dimension of `movie$inputs`.

n_players *Number of players (controller ports) in a parsed movie*

Description

Number of players (controller ports) in a parsed movie

Usage

```
n_players(movie)
```

Arguments

movie	A replayr_movie (returned by read_movie() or any of the format-specific readers).

Value

Integer scalar: the second dimension of movie$inputs.

player *Per-frame button presses for one player*

Description

Per-frame button presses for one player

Usage

```
player(movie, idx = 1L)
```

Arguments

movie	A replayr_movie (returned by read_movie() or any of the format-specific readers).
idx	Player index, 1-based. Defaults to 1 (player 1).

Value

Logical matrix, n_frames(movie) rows by n_buttons(movie) columns. Column names are the universal-button mnemonics.

Examples

```
## Not run:
  mv <- read_movie("data/raw_nes/super_mario_bros/1349.fm2")
  p1 <- player(mv)
  colSums(p1)        # number of frames each button was held

## End(Not run)
```

press_counts *Press counts per universal-button slot*

Description

Press counts per universal-button slot

Usage

```
press_counts(movie, idx = 1L)
```

Arguments

movie	A replayr_movie (returned by `read_movie()` or any of the format-specific readers).
idx	Player index, 1-based.

Value

Named integer vector of length `n_buttons(movie)`. Names are the universal-button friendly labels.

read_bk2 *Read a BizHawk .bk2 movie file*

Description

.bk2 is a ZIP archive whose `Input Log.txt` describes inputs and whose `Header.txt` carries metadata (Author, Core, Platform, GameName, SHA1, ...). The parser uses the LogKey line to map column positions to universal-button slots, supporting NES, SNES, Atari 2600, Game Boy / GBC / GBA, N64, PSX in one pass. Without a LogKey it falls back to NES positional UDLRTSBA mapping.

Usage

```
read_bk2(path)
```

Arguments

path	Path to a .bk2 file.

Details

Analog-stick columns (N64 / PSX) are quantised to 8 directions onto slots 21..28; the c-stick / right stick goes onto slots 17..20.

Value

A `replayr_movie` object.

read_dtm *Read a Dolphin .dtm GameCube movie file*

Description

13 GC buttons map onto universal slots 1..14 alongside their NES/SNES counterparts. The analog stick is quantised to 8 cardinal/intercardinal directions (slots 21..28); the C-stick is quantised more coarsely onto the four C-button slots (17..20). L/R analog triggers fire the corresponding shoulder slot when pulled past 50%. Output is 28 buttons wide. Wii-mode .dtm files raise an error.

Usage

```
read_dtm(path)
```

Arguments

path	Path to a .dtm file.

Value

A `replayr_movie` object.

read_fcm *Read a FCEU .fcm movie file*

Description

Parses the binary delta-encoded body into the standard `replayr_movie` shape. Output is 4 player slots x 8 button slots wide; nearly all FCM publications use only player 1 but the format reserves four ports.

Usage

```
read_fcm(path)
```

Arguments

path	Path to a .fcm file.

Value

A `replayr_movie` object.

read_fm2 *Read an FCEUX .fm2 movie file*

Description

Parses the text header into a list and the input log into a logical 3D array indexed `(frame, player, button)`. Buttons are positional in RLDUTSBA order so the output's third dimension lines up with the first eight slots of the universal layout.

Usage

```
read_fm2(path)
```

Arguments

`path`	Path to a .fm2 file.

Value

A `replayr_movie` object.

read_gmv *Read a Gens-rerecording .gmv movie file*

Description

Genesis controllers map onto universal slots 1..4 (d-pad), 5 (Start), 7/8 (B/A; Genesis has no Select), 9/10 (X/Y), 13 (C), 14 (Z), 15 (Mode). Output is 15 buttons wide so every Genesis-relevant column sits at its canonical universal slot.

Usage

```
read_gmv(path)
```

Arguments

`path`	Path to a .gmv file.

Value

A `replayr_movie` object.

`read_lmp` *Read a Doom .lmp demo file*

Description

Parses a vanilla Doom 1.9 `.lmp` demo into the standard `replayr_movie` shape. Returns one row per game tic (per the active-player count from `playeringame[]`).

Usage

```
read_lmp(path)
```

Arguments

`path`	Path to a `.lmp` file.

Details

Doom records the resulting `ticcmd_t`, not the physical device events. Keyboard, mouse and joystick input may all contribute to its signed movement fields. The reader therefore exposes inputs two ways:

- Universal `inputs` array: action buttons map to the NES-shaped layout (ATTACK -> slot 8 "A", USE -> slot 7 "B"), and the **sign** of forwardmove / sidemove drives the D-pad slots (forwardmove > 0 -> U, < 0 -> D; sidemove > 0 -> R, < 0 -> L). This makes a Doom keypress or other movement command comparable with digital gamepad data; it must not be interpreted as evidence that a keyboard key was held.
- Original analog axes are preserved as `n_frames x 4` integer matrices in `header$forwardmove`, `header$sidemove`, and `header$angleturn`. Weapon-switch state lives in `header$weaponselect` and BT_SPECIAL flags in `header$has_special`.

`angleturn` is intentionally **not** thresholded onto a button slot, because its magnitude contains useful turning information that a binary projection would discard.

Boom / PrBoom+ extended demos (leading 0xFF byte) are not yet supported; this reader will error out with a clear message if the file uses that variant.

Value

A `replayr_movie` object whose `header[["_format"]]` is `"lmp"`, with extra header columns documented above.

See Also

`read_movie()`, `supported_formats()`

read_lsmv *Read an lsnes .lsmv movie file*

Description

.lsmv is a ZIP archive containing an `input` text member plus various metadata members (gametype, controlsversion, coreversion, systemid, rom.sha256, projectid, rerecords, ...). Currently supports the SNES standard gamepad (12-char masks; 16-char with the lsnes 4-col extension suffix) and Game Boy / GBC (8-char masks via libgambatte). Other port types (mouse, superscope, justifier) raise an error for now.

Usage

```
read_lsmv(path)
```

Arguments

`path`	Path to an .lsmv file.

Details

Super Game Boy recordings (gametype starting with `gambatte_sgb`) carry a 12-char SNES-shaped mask but only ever fire the first eight positions – they're folded down to the GB shape (8 buttons) so SGB and GB pubs share a single in-memory width.

Value

A `replayr_movie` object.

read_m64 *Read a Mupen64 .m64 movie file*

Description

The N64 controller has 14 buttons + an analog stick. Buttons map onto universal slots 1..14 (with C-buttons at 17..20); the analog stick is quantised to 8 directions and routed to slots 21..28. Output is 28 buttons wide. Stick deflections below ~12.5% magnitude are treated as neutral. Multi-controller M64 streams are rejected because the format does not store controller identifiers: samples occur in the order the game polls its connected controllers, which cannot be reconstructed reliably without running the game.

Usage

```
read_m64(path)
```

Arguments

`path`	Path to a .m64 file.

Value

A `replayr_movie` object.

read_movie *Read any supported movie file by extension*

Description

Dispatches to the appropriate reader based on the file extension: .fm2 -> `read_fm2()`, .bk2 -> `read_bk2()`, .fcm -> `read_fcm()`, .lsmv -> `read_lsmv()`, .gmv -> `read_gmv()`, .vbm -> `read_vbm()`, .m64 -> `read_m64()`, .smv -> `read_smv()`, .pxm -> `read_pxm()`, .pjm -> `read_pjm()`, .dtm -> `read_dtm()`, .lmp -> `read_lmp()`.

Usage

```
read_movie(path)
```

Arguments

`path`	Path to a movie file.

Details

Returns a `replayr_movie` regardless of source format. The `header[["_format"]]` field records which parser ran. The `inputs` array is `(n_frames, n_players, n_buttons)`; `n_buttons` varies per platform (8 NES, 12 SNES, 15 Genesis, 28 N64/GC, 30 PSX).

Value

A `replayr_movie` object.

See Also

`supported_formats()`

read_pjm *Read a PSXjin .pjm movie file*

Description

Parses the PSXjin movie format into the standard `replayr_movie` shape. Supports the text body in its one-controller (20-byte line), two-controller (34-byte line) and analog (wider line) variants, plus the binary 5-byte-per-frame body. The PSX gamepad maps onto universal slots 1..12 (NES + SNES face / shoulder layout) plus 29 / 30 for L2 / R2; the output is 30 buttons wide. Two-controller movies yield a two-player movie (analysis code that wants player 1 simply reads the first player).

Usage

```
read_pjm(path)
```

Arguments

`path`	Path to a .pjm file.

Value

A `replayr_movie` object.

`read_pxm` *Read a PCSX-rerecording .pxm movie file*

Description

Reads standard digital and analog-controller PXM movies. Button input maps to the 30-slot PSX universal layout; analog sticks are quantised in the same way as other replayr PSX readers. The final per-frame control byte is exposed through `movie$commands`. Mouse, NegCon, and light-gun controller types are rejected instead of being silently mis-decoded.

Usage

```
read_pxm(path)
```

Arguments

`path`	Path to a .pxm file.

Value

A `replayr_movie` object.

`read_smv` *Read a Snes9x-rerecording .smv movie file*

Description

SNES gamepads come out 12 buttons wide (universal slots 1..12) so .smv publications share the same shape as .lsmv and .bk2 SNES pubs. The controller mask at offset 0x14 selects which ports the file polls; each polled port becomes one player in the output.

Usage

```
read_smv(path)
```

Arguments

`path`	Path to a .smv file.

Value

A `replayr_movie` object.

read_vbm *Read a VBA-rerecording .vbm movie file*

Description

GB, GBC, and SGB publications come out 8 buttons wide (universal slots 1..8). GBA publications come out 12 buttons wide (slots 1..12) so the L/R shoulders land at their canonical universal positions. SGB multiplayer controller streams are returned with one player dimension per active controller flag.

Usage

```
read_vbm(path)
```

Arguments

path	Path to a .vbm file.

Value

A replayr_movie object.

replayr_movie *The* replayr_movie *S3 class*

Description

The canonical return type of every reader in replayr. A replayr_movie is a list with the following elements:

Details

header Named list of header key-value pairs lifted from the source file. The synthetic _format entry records which parser produced the movie (one of "fm2", "bk2", "fcm", "lsmv", "gmv", "vbm", "m64", "smv", "pxm", "pjm", "dtm", "lmp").

commands Integer vector of length n_frames. Per-frame commands byte (Reset, Power, format-specific bits). Most TAS frames are 0.

inputs Logical 3D array, dimensions (n_frames, n_players, n_buttons). TRUE means the button was held that frame.

source_path Path the movie was loaded from, or NA.

button_mnemonics Character vector. One single-character mnemonic per third-dimension slot of inputs.

button_names Character vector. Friendly labels paired with button_mnemonics.

Use n_frames(), n_players(), n_buttons() to read the array shape; player() to extract one player's frames as a 2D matrix; press_counts() to summarise hold-counts per button.

See Also

read_movie(), universal_buttons()

supported_formats *Names of the supported movie file extensions*

Description

Names of the supported movie file extensions

Usage

```
supported_formats()
```

Value

Character vector of file-extensions (lowercase, with leading dot).

Examples

```
supported_formats()
```

universal_buttons *Universal button mnemonics*

Description

One-character symbol per universal-button slot. Length 40. Indexed by slot number using one-based R semantics: slot 1 = "R" (Right), slot 8 = "A", and so on. Slots 31..40 are the Intellivision / ColecoVision numeric keypad (digits 0-9). INTV's Enter/Clear and Coleco's Star/Pound do not have dedicated slots — they fold into Start (slot 5) and Select (slot 6) since they're functionally Start/Select on those platforms (pressed once at boot).

Usage

```
universal_buttons()
```

Details

Value

Character vector of length 40.

See Also

`universal_button_names()`

Examples

```
universal_buttons()
```

`universal_button_names` *Universal button friendly names*

Description

Long, human-readable label per slot. Same length as `universal_buttons()` and indexed identically.

Usage

```
universal_button_names()
```

Value

Character vector of length 40.

Index

∗ **internal**
    replayr-package, 2

n_buttons, 3
n_buttons(), *12*
n_frames, 3
n_frames(), *12*
n_players, 4
n_players(), *12*

player, 4
player(), *12*
press_counts, 5
press_counts(), *12*

read_bk2, 5
read_bk2(), *10*
read_dtm, 6
read_dtm(), *10*
read_fcm, 6
read_fcm(), *10*
read_fm2, 7
read_fm2(), *10*
read_gmv, 7
read_gmv(), *10*
read_lmp, 8
read_lmp(), *10*
read_lsmv, 9
read_lsmv(), *10*
read_m64, 9
read_m64(), *10*
read_movie, 10
read_movie(), *3–5*, *8*, *12*
read_pjm, 10
read_pjm(), *10*
read_pxm, 11
read_pxm(), *10*
read_smv, 11
read_smv(), *10*
read_vbm, 12
read_vbm(), *10*
replayr *(*replayr-package*)*, 2
replayr-package, 2
replayr_movie, *3–5*, 12

supported_formats, 13
supported_formats(), *8*, *10*

universal_button_names, 14
universal_button_names(), *2*, *13*
universal_buttons, 13
universal_buttons(), *2*, *3*, *12*, *14*